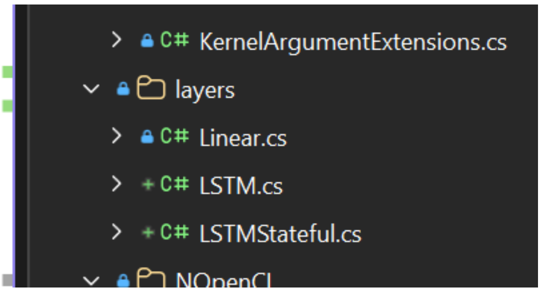
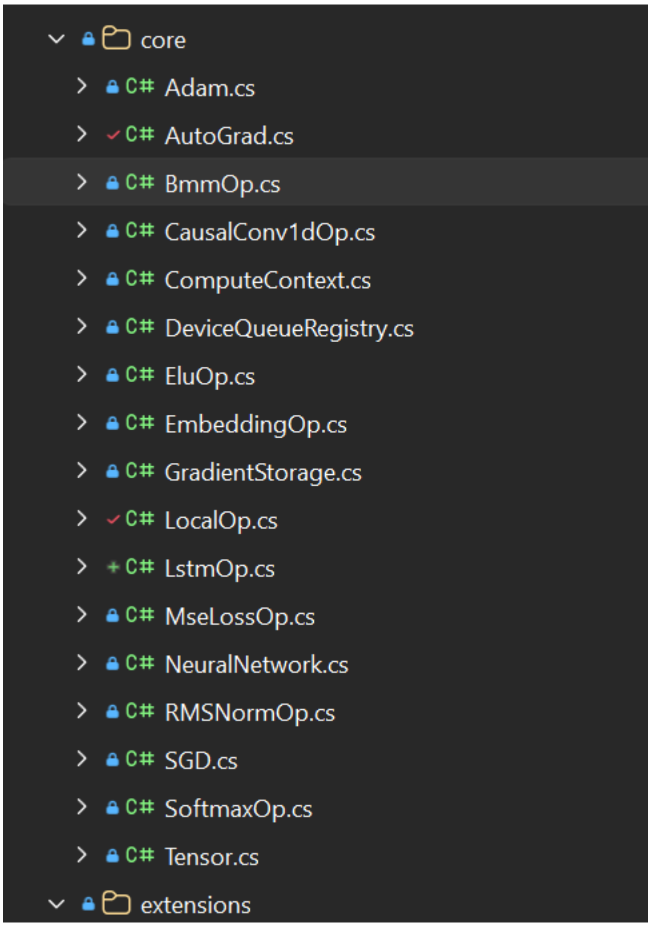
```
=== ЗАПУСК ТЕСТА АВТОНОМНОГО СЛОЯ LSTMSTATEFUL ===
максимальные размеры локальной группы для устройства на основе варпа:X=256,Y=4
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-78f6df.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-b520fc.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-9ebfb6.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-d4fe4f.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-22954d.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-31b387.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-4224bc.d

[ШАГ 1]: Прямой проход Чанка 1...
-> Чанк 1 успешно обработан слоем.

[ШАГ 2]: Прямой проход Чанка 2...
-> Чанк 2 успешно обработан слоем.

[ШАГ 3]: Вычисление ошибки MSE Loss на выходе Чанка 2...
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-41d26b.d
-> Текущий MSE Loss на Чанке 2: 0,879065

[ШАГ 4]: Запуск сквозного обратного прохода (AutoGrad.Backward)...
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-c082bf.d
Could not open file: C:\Users\╨Ф╨╝╨╕╤В╤А╨╕╨╣\AppData\Local\Temp\dep-014c72.d
-> Каскадный обход BPTT успешно завершен.

=== АНАЛИЗ ПЕРЕТЕКАНИЯ ГРАДИЕНТОВ В LSTMSTATEFUL ===
1. Градиент дотек до данных Чанка 2 (gradX2): True
2. Градиент перешагнул барьер чанков и дотек до Чанка 1 (gradX1): False
3. Градиент для объединенных весов WeightW создан: True
[Успех] Значение dL/dW (первый элемент матрицы весов): 0,0000E+000

=== ТЕСТ LSTMSTATEFUL УСПЕШНО ЗАВЕРШЕН ===



C:\Users\Дмитрий\github_repo\my_pytchon_opencl_lib\csharp\cstorchcl\LSTMTest\bin\Debug\net10.0\LSTMTest.exe (процесс 20484) завершил работу с кодом 0 (0x0).
Чтобы автоматически закрывать консоль при остановке отладки, включите параметр "Сервис" ->"Параметры" ->"Отладка" -> "Автоматически закрыть консоль при остановке отладки".
Нажмите любую клавишу, чтобы закрыть это окно:
```

С пока все , давай проведем обзор что в ббилотеку уже есть , и что может быть полезно реализовать.
Слои :



- KernelArgumentExtensions.cs
- layers
  - Linear.cs
  - LSTM.cs
  - LSTMStateful.cs
- NOpenCL

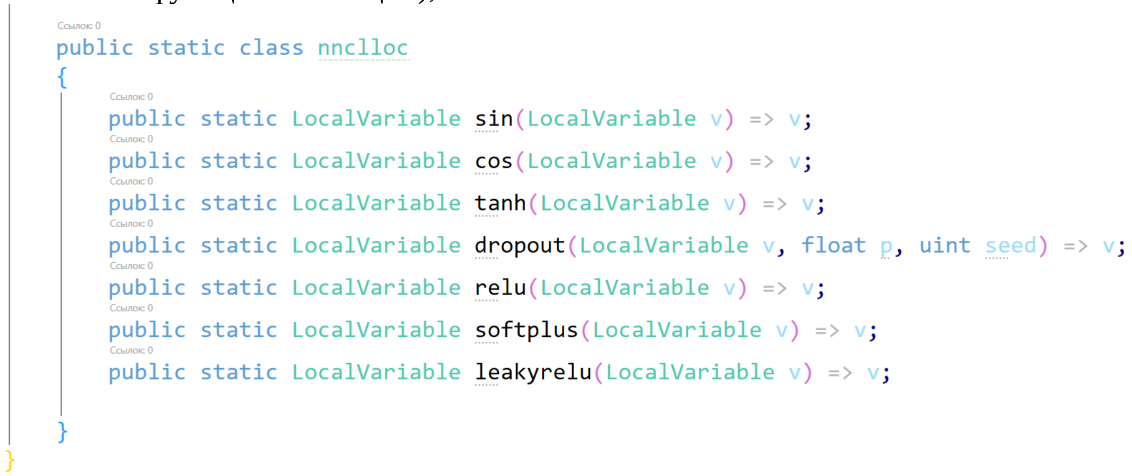
Линейный и lstm.
Оптимизаторы:



- core
  - Adam.cs
  - AutoGrad.cs
  - BmmOp.cs
  - CausalConv1dOp.cs
  - ComputeContext.cs
  - DeviceQueueRegistry.cs
  - EluOp.cs
  - EmbeddingOp.cs
  - GradientStorage.cs
  - LocalOp.cs
  - LstmOp.cs
  - MseLossOp.cs
  - NeuralNetwork.cs
  - RMSNormOp.cs
  - SGD.cs
  - SoftmaxOp.cs
  - Tensor.cs
- extensions

Стохстический и adam.
Глобальные объекты:
Softmax (поддерживает редукцию только по последне оси (а бывают ситуации когда нужэно по другим?)),RmsNorm,Permute,Bmm,CausalConv1d
(возможно в будущем поробую релизовать conv2d для работы с изображенями),embedding,

elu (только для отладки, в будщем возможна ее реализация в рамках localrunner , который отвечает за функции активации),MSE.



```
Ссылок: 0
public static class nnclloc
{
    Ссылок: 0
    public static LocalVariable sin(LocalVariable v) => v;
    Ссылок: 0
    public static LocalVariable cos(LocalVariable v) => v;
    Ссылок: 0
    public static LocalVariable tanh(LocalVariable v) => v;
    Ссылок: 0
    public static LocalVariable dropout(LocalVariable v, float p, uint seed) => v;
    Ссылок: 0
    public static LocalVariable relu(LocalVariable v) => v;
    Ссылок: 0
    public static LocalVariable softplus(LocalVariable v) => v;
    Ссылок: 0
    public static LocalVariable leakyrelu(LocalVariable v) => v;

}
}
```

Функции GELU нет . Можешь описать ее теоретические аспекты?
Также интересует теория , как можно сохранить веса в бинарный ФОРМАТ (для начала в .pth подобный , так как safetensors это архив и сним будет сложнее) и соотвевенно прочитать их из бинарного тформата.(чтобы можно было сохранить результат обучения).
Также расскажит про соттовные части multi head attenction и можно ли их скомпоновать в единое pencl ядро как с lstm?